RAPPORT DE TP3 ET EXERCICE 3

Travaux de programmation et modélisation

Partie I : gestion de résidence
Partie II : gestion du personnel

Travail pratique	TP3 - Exercice 3
Groupe	Groupe 12
Formation	MIA FAST NATI

Rapport établi à partir des fichiers source et des diagrammes présents dans le dossier du projet.

Résumé

Ce rapport rassemble deux travaux distincts du dossier du groupe : le TP3 consacré à la gestion d’une résidence étudiante et l’Exercice 3 consacré à la gestion du personnel. La première partie présente les besoins, les diagrammes UML et le programme de résidence ; la seconde décrit l’héritage et le polymorphisme dans le programme des employés. Les deux exercices sont étudiés séparément, car ils possèdent des objectifs, des fichiers source et des commandes de compilation différents.

Sommaire

• PARTIE I - TP3 : Gestion de résidence
• 1. Présentation et objectifs
• 2. Besoins fonctionnels et acteurs
• 3. Modèle de données et diagramme de classes
• 4. Déroulement d’une attribution
• 5. Cycle de vie d’une demande
• 6. Réalisation, limites et compilation
• PARTIE II - Exercice 3 : Gestion du personnel
• 1. Objectif et diagramme de classes
• 2. Héritage, polymorphisme et exécution
• Conclusion générale

1. Présentation et objectifs

La disponibilité des chambres, les dossiers des étudiants et les périodes d'occupation doivent être suivis de manière cohérente. Le projet propose un petit outil en mode texte qui centralise ces opérations et rend consultable l'état des chambres et des demandes.

Objectifs du travail

- Modéliser les principaux acteurs, les données et les états du processus d'hébergement.
- Gérer les enregistrements d'étudiants, de chambres, de demandes et d'attributions.
- Permettre le suivi des occupations en cours, des départs et de l'historique.
- Calculer des indicateurs simples sur l'occupation de la résidence.

Périmètre

La partie I porte sur le système de gestion de résidence et son programme TP3_Groupe12_MIA_FAST.cpp. La partie II, distincte, est consacrée à l'exercice de programmation orientée objet sur les employés, dans Exercice3_Groupe12_MIA_FAST.cpp. Les deux travaux sont présentés dans ce même PDF, sans confondre leurs modèles ni leurs exécutables.

2. Besoins fonctionnels et acteurs

Le diagramme de cas d'utilisation identifie deux acteurs. Le responsable réalise les opérations de gestion et de suivi. L'étudiant peut soumettre une demande et consulter les chambres. L'attribution inclut une vérification de disponibilité.

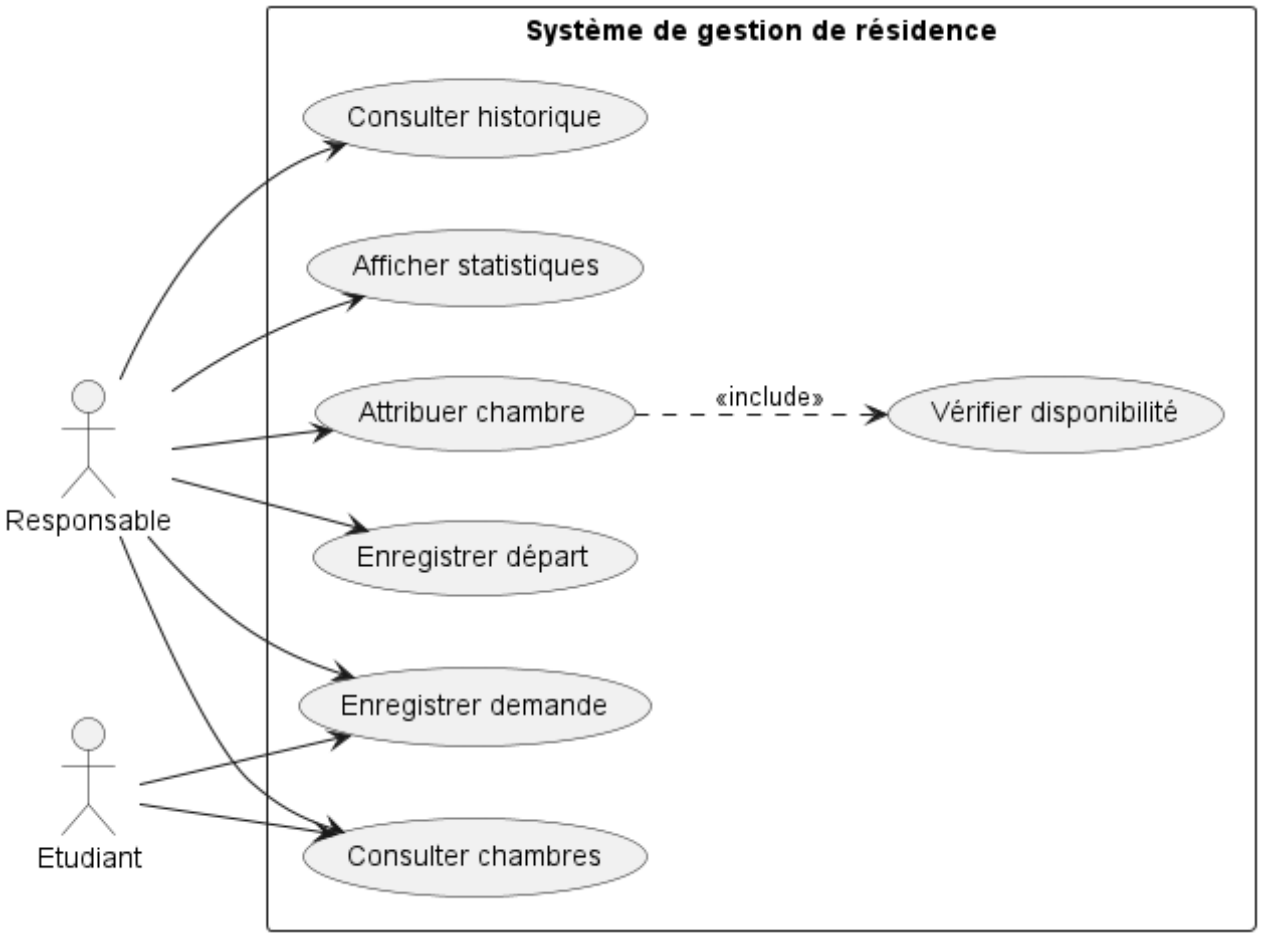


Figure 1 - Cas d’utilisation du système de gestion de résidence

Principales fonctionnalités

Domaine	Comportement fourni par le programme
Étudiants	Enregistrer un étudiant avec un identifiant généré, son nom complet et son adresse courriel ; afficher la liste.
Chambres	Enregistrer une chambre avec numéro, bâtiment et capacité ; afficher toutes les chambres, les chambres libres ou occupées.
Demandes	Créer une demande associée à un étudiant et à une date ; consulter les demandes et leur état.
Attributions et départs	Affecter une chambre libre à une demande en attente ; enregistrer ensuite le départ et libérer la chambre.
Suivi	Afficher les occupations actives, l’historique et les indicateurs d’occupation.

3. Modèle de données et diagramme de classes

Le diagramme de classes représente les concepts métier suivants : résidence, bâtiment, chambre, étudiant, demande d'hébergement et attribution. Deux énumérations décrivent les états possibles d'une chambre et d'une demande. Les associations expriment notamment qu'un étudiant peut déposer plusieurs demandes et avoir plusieurs attributions au fil du temps, tandis qu'une attribution concerne une chambre.

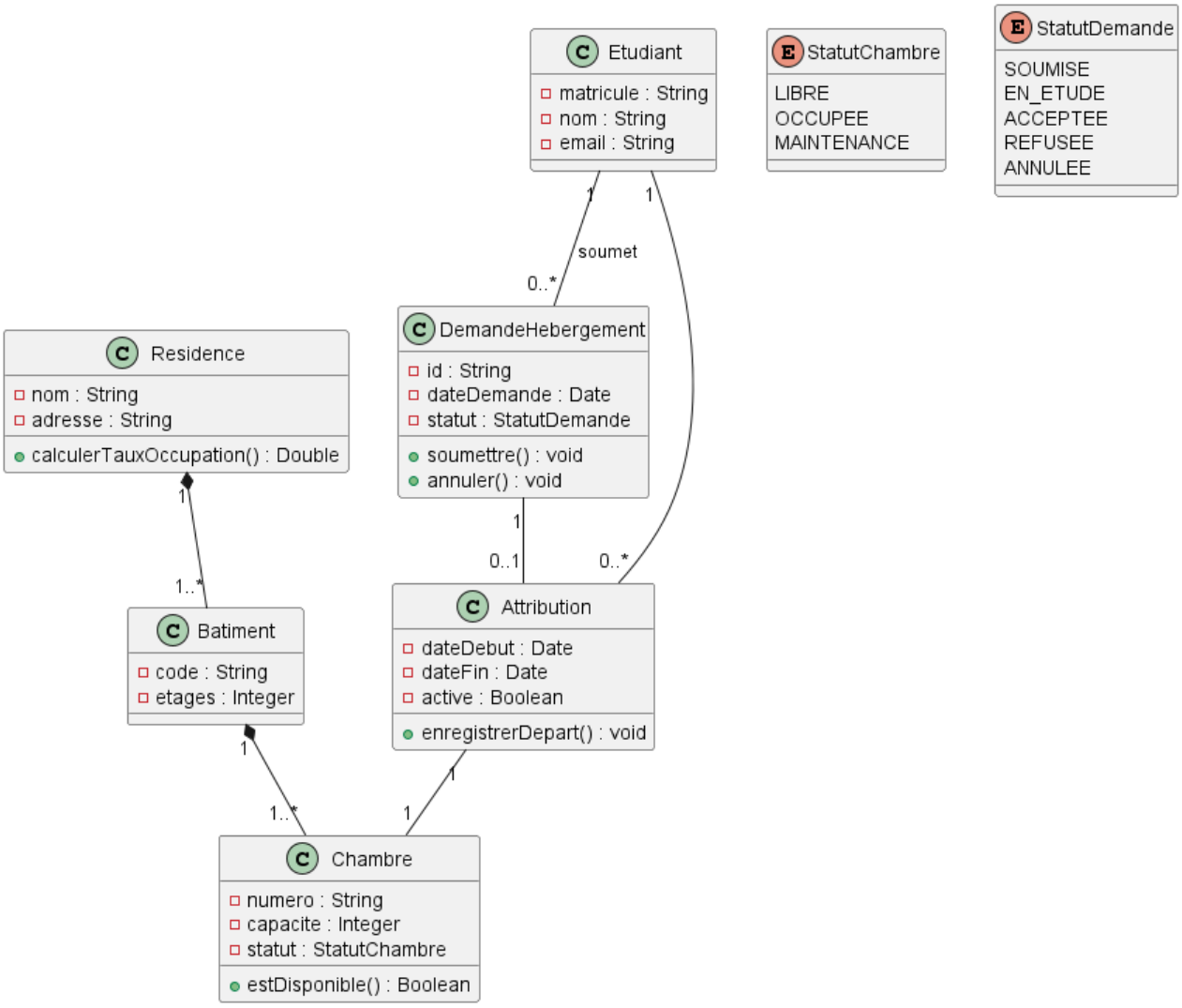


Figure 2 - Modèle de classes UML de la gestion de résidence

Correspondance avec le code

Le code source utilise des structures Etudiant, Chambre, DemandeHebergement et Attribution, stockées dans des tableaux séparés. Les identifiants numériques relient ces enregistrements. Il n'y a pas d'objets Residence ou Batiment distincts dans l'implémentation : le nom du bâtiment est directement enregistré dans chaque chambre. Le diagramme est donc une vue conceptuelle plus riche que la structure réellement programmée.

Données conservées

Le programme réserve des tableaux de taille fixe : jusqu'à 100 étudiants, 100 chambres, 200 demandes et 200 attributions. Les numéros sont générés à partir du nombre d'éléments déjà enregistrés. Les informations sont conservées uniquement en mémoire pendant l'exécution.

4. Déroulement d'une attribution

Le diagramme de séquence présente l'échange entre le responsable, l'application, la résidence, une chambre et une attribution. Il distingue le cas où une chambre est disponible du cas où aucune place n'est disponible.

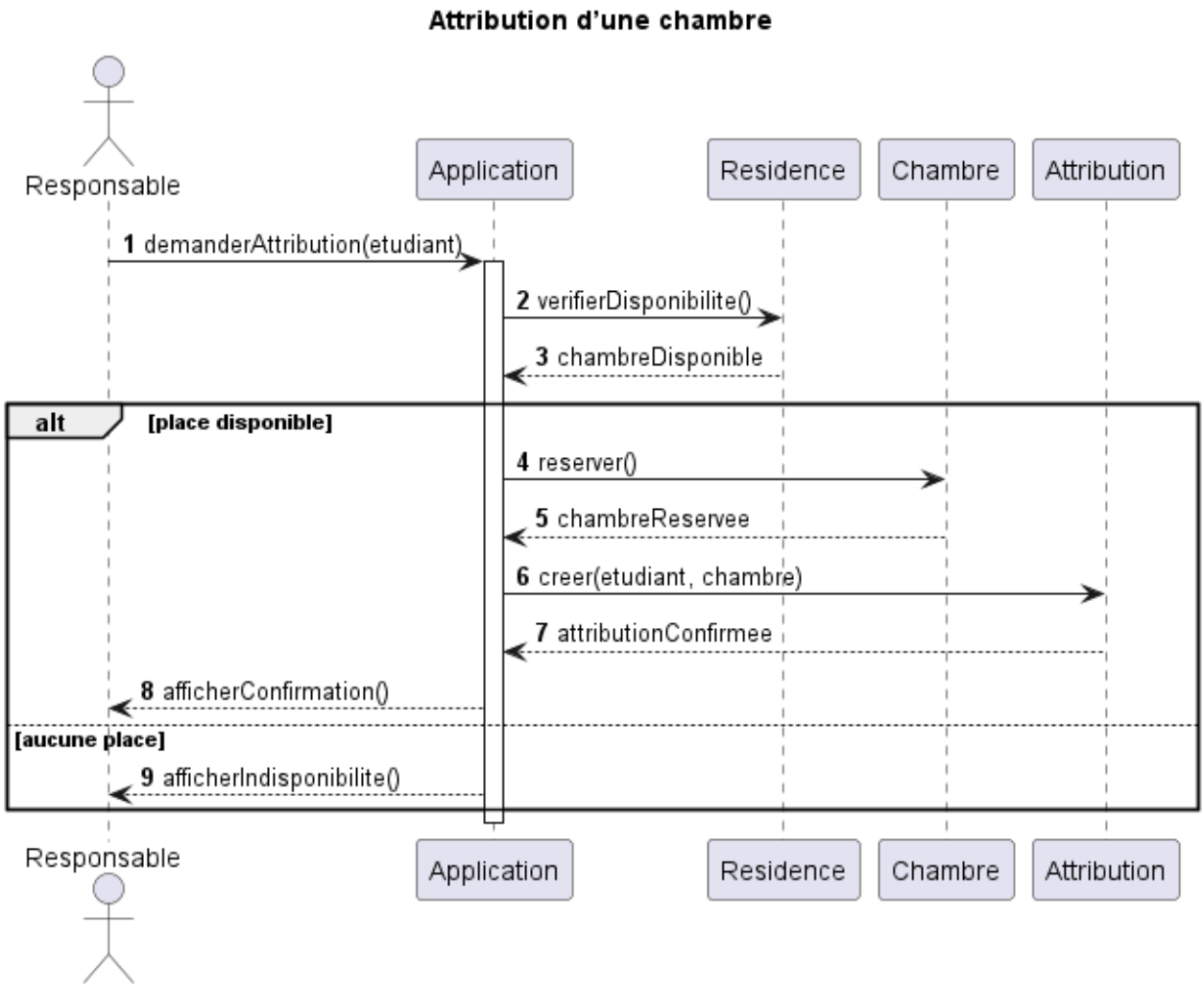


Figure 3 - Séquence conceptuelle d'attribution d'une chambre

Dans le programme, l'attribution vérifie successivement que les données nécessaires existent, que l'étudiant ne possède pas déjà une chambre active, que la demande appartient bien à cet étudiant et reste en attente, puis que la chambre choisie est connue et libre. En cas de succès, le programme crée une attribution active, marque la chambre occupée et passe la demande à l'état accepté.

5. Cycle de vie d'une demande

Le diagramme d'états montre les transitions prévues : soumission, prise en charge, acceptation avec attribution ou refus, puis fin du processus. Une demande refusée peut ensuite être annulée selon le diagramme.

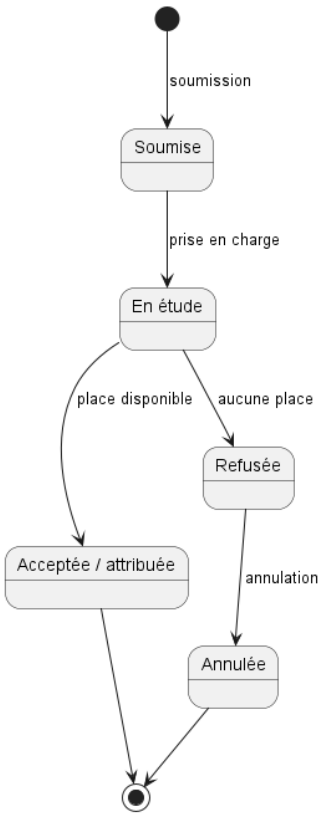


Figure 4 - États et transitions prévus pour une demande d’hébergement

6. Réalisation du programme

Le fichier principal est un programme en mode console. Malgré son extension .cpp, sa structure est procédurale : il utilise les bibliothèques C standard, des structures de données, des tableaux globaux et des fonctions. Le menu principal propose treize opérations et une option de sortie.

Organisation des données et opérations

Module	Fonctions principales
Étudiants	enregistrerEtudiant, afficherEtudiants
Chambres	enregistrerChambre, afficherChambres, afficherChambresLibres, afficherChambresOccupees
Demandes	enregistrerDemande, afficherDemandes
Attributions	attribuerChambre, enregistrerDepart, afficherAttributionsActives, afficherHistorique
Statistiques	afficherStatistiques

Règles appliquées

- Une demande ne peut être créée que si un étudiant est déjà enregistré.
- L'attribution exige un étudiant, une chambre et une demande existants.
- Une chambre occupée ne peut pas être attribuée une seconde fois.
- Un étudiant ne peut pas avoir plus d'une attribution active à la fois.
- Le départ clôt l'attribution, libère la chambre et marque la demande comme terminée.

Calcul des statistiques

Le programme calcule la proportion de chambres occupées parmi les chambres enregistrées, puis une proportion de places occupées à partir des capacités déclarées. Dans le code, toutes les places d'une chambre sont comptées comme occupées dès que cette chambre est affectée, même si une seule personne y est associée. Le taux des places ne reflète donc pas nécessairement le nombre réel d'étudiants hébergés.

7. Limites et améliorations possibles

Point à améliorer	Observation et piste d'évolution
Persistance	Les données disparaissent à la fermeture ; ajouter une sauvegarde dans des fichiers ou une base de données.
État refusé	Le diagramme prévoit un refus, mais le programme ne propose pas d'opération de refus et n'affecte jamais cet état.
Annulation	Le diagramme prévoit l'annulation d'une demande, mais aucune fonction du menu ne l'implémente.
Capacité	Le code affecte une chambre entière à un étudiant ; il faudrait préciser si la capacité correspond à plusieurs places ou à une seule attribution.
Saisie	Les dates sont de simples chaînes de caractères et les saisies numériques ne sont pas systématiquement contrôlées.
Modèle UML	Le diagramme distingue la résidence et les bâtiments, alors que le programme stocke le bâtiment sous forme de texte dans la chambre.

8. Compilation et lancement

Le fichier de gestion de résidence peut être compilé depuis le dossier du projet avec g++. Le code n’a pas été exécuté dans le cadre de la rédaction de ce rapport ; les commandes ci-dessous reprennent le mode d’emploi indiqué dans le README du projet.

Depuis PowerShell, dans le dossier du projet :

```
g++ -std=c++11 TP3_Groupe12_MIA_FAST.cpp -o residence.exe
.\residence.exe
```

Après le lancement, le menu s’affiche. Saisir le numéro de l’opération, puis suivre les demandes de saisie du programme. Pour quitter, choisir 0.

Conclusion de la partie I

Le TP3 couvre les étapes essentielles du suivi d’hébergement : enregistrement des étudiants et des chambres, dépôt de demandes, attribution, départ et consultation de statistiques. Les diagrammes UML offrent une vue fonctionnelle et dynamique du système, tandis que le programme fournit une première réalisation en console. Les améliorations prioritaires sont de rapprocher le modèle et le code, de corriger le calcul des places occupées et d’ajouter la persistance ainsi que les transitions de refus et d’annulation.

PARTIE II - EXERCICE 3

Gestion du personnel par héritage et polymorphisme

Cette partie est un exercice séparé du TP3 de résidence. Elle met en pratique la programmation orientée objet en C++ à travers une classe abstraite Employe et trois spécialisations : Enseignant, AgentAdministratif et Technicien. Le programme crée un objet de chaque type, calcule son salaire, puis affiche ses informations.

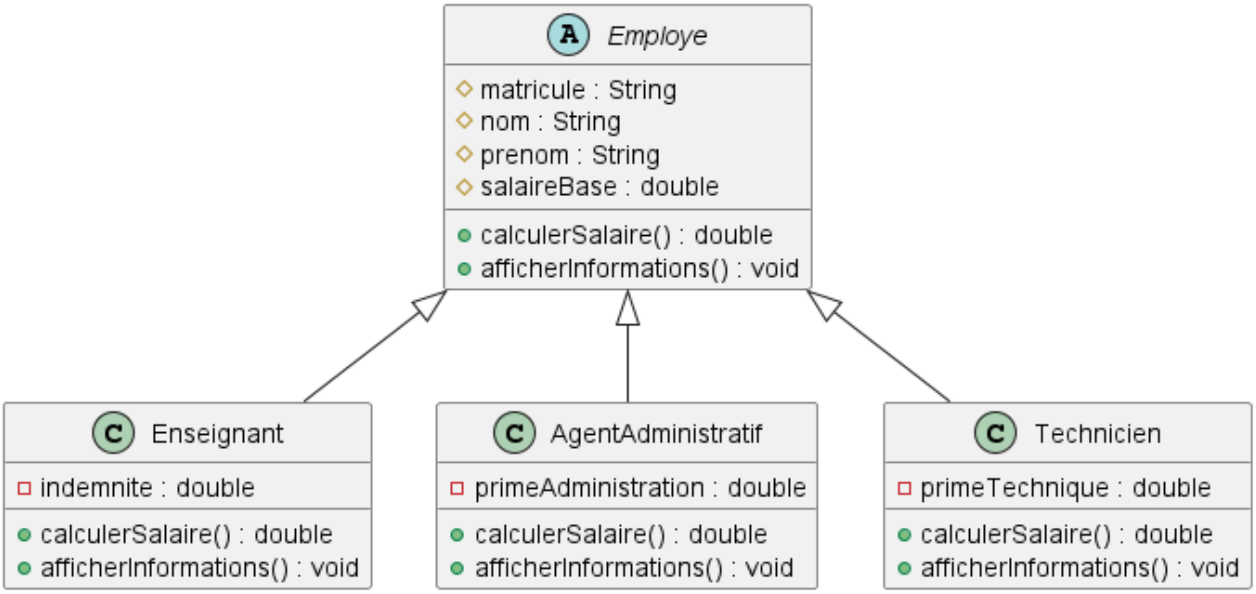


Figure 5 - Diagramme de classes de l'exercice 3 sur les employés

Structure et principes utilisés

La classe Employe regroupe les données communes (matricule, nom, prénom et salaire de base). Sa méthode calculerSalaire() est virtuelle pure, ce qui rend la classe abstraite : on ne crée pas directement un employé générique. Chaque classe dérivée fournit sa propre formule et redéfinit l'affichage.

Formules de salaire

Catégorie	Calcul implémenté	Valeur avec les données du programme
Enseignant	Salaire de base + indemnité	400 000 + 75 000 = 475 000
Agent administratif	Salaire de base + prime administrative	300 000 + 50 000 = 350 000
Technicien	Salaire de base + prime technique	350 000 + 60 000 = 410 000

Les valeurs sont reprises des objets créés dans le code. Aucune unité monétaire n'est précisée dans les fichiers du projet.

Polymorphisme et gestion mémoire

Dans main(), le programme stocke les trois catégories dans un vecteur de pointeurs intelligents unique_ptr<Employe>. La boucle appelle afficherInformations() sur chaque élément. Grâce aux méthodes virtuelles, C++ exécute la version propre à la catégorie réelle de l'objet ; le calcul du salaire suit le même mécanisme. Le destructeur virtuel permet une destruction correcte des objets dérivés à travers un pointeur vers la classe de base.

Compilation et lancement de l’exercice 3

Cet exercice requiert C++14, notamment pour make_unique. Depuis PowerShell, dans le dossier contenant le fichier :

```
g++ -std=c++14 Exercice3_Groupe12_MIA_FAST.cpp -o employes.exe
.\employes.exe
```

Conclusion générale

Les deux travaux illustrent des aspects complémentaires du développement logiciel. Le TP3 propose une gestion fonctionnelle des hébergements avec des diagrammes UML et un programme en console structuré de manière procédurale. L’Exercice 3 met l’accent sur les classes, l’héritage, les méthodes virtuelles et le polymorphisme. Les garder dans deux parties distinctes clarifie leurs objectifs et facilite la compilation de chaque programme séparément.

Éléments du dossier utilisés

Partie I : TP3_Groupe12_MIA_FAST.cpp, diagrammes UML de cas d’utilisation, de classes, de séquence et d’états-transitions, et README. Partie II : Exercice3_Groupe12_MIA_FAST.cpp et son diagramme de classes.